

TECNOLÓGICO NACIONAL DE MÉXICO

CAMPUS MONCLOVA

Materia: Fundamentos Bases de Datos

Actividad U.6 – Tarea 6.2

Maestro: Rubén Miguel Riojas Rodríguez

Semestre: 5to Semestre

Alumno: José Luis Luna García

Matricula: I24050385

Tabla de contenido

Unidad 6 - Tarea 6.2 3

Implementación de script DDL, carga de datos y consultas DML 3

Ficha Técnica y Entorno de Operación del SGBD 3

Definición del Esquema e Integridad Relacional (DDL) 4

Carga y Poblado de Datos (DML) 7

Consultas Analíticas y Extracción de Datos (Parte A - DQL) 12

Operaciones de Mantenimiento y Validación de Reglas de Integridad (Parte B) . 16

Conclusiones y Evaluación Técnica 20

Unidad 6 - Tarea 6.2

Implementación de script DDL, carga de datos y consultas DML

Ficha Técnica y Entorno de Operación del SGBD

El presente reporte técnico documenta formalmente la entrega de la Actividad en donde han sido integrandas las evidencias paso a paso solicitadas: verificación previa de inexistencia de la base de datos en el servidor, arrastre e inspección de scripts en SQL Server Management Studio (SSMS), ejecución DDL con restricciones de integridad, comprobación de tablas vacías, carga masiva DML, y la ejecución analítica exhaustiva de las 8 consultas requeridas (4 de extracción DQL y 4 de mantenimiento e integridad referencial).

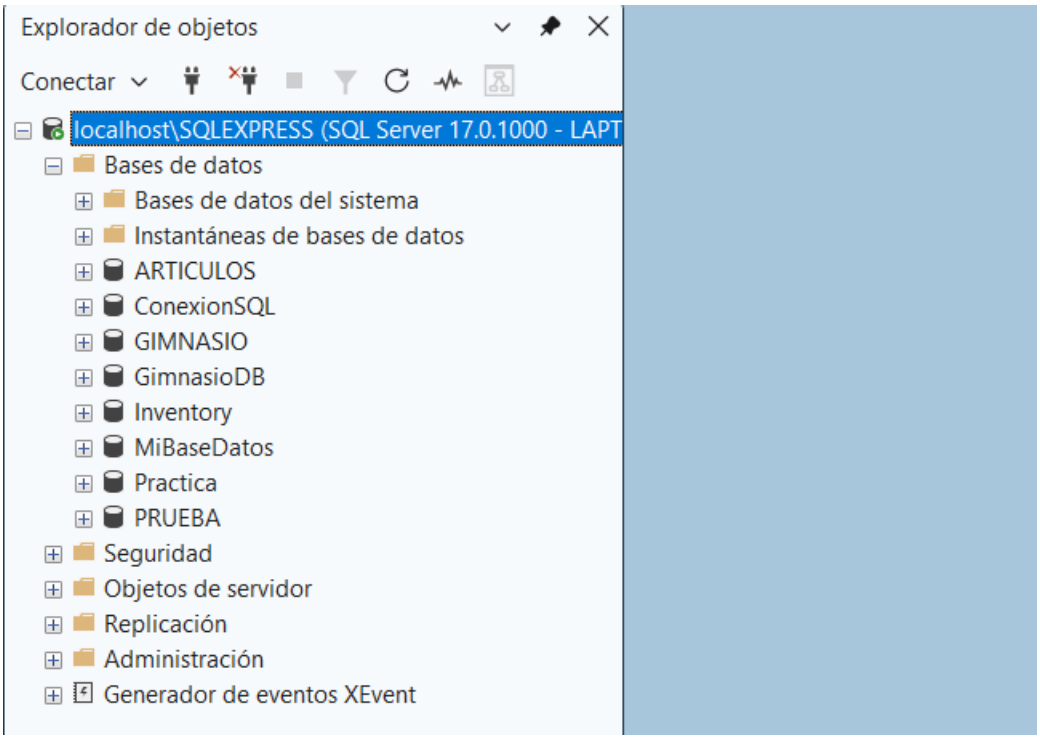
Todo el desarrollo, ejecución de sentencias Transact-SQL y comprobación de resultados se llevó a cabo en una estación de trabajo bajo Microsoft SQL Server Management Studio (SSMS) conectada a la instancia local localhost\SQLEXPRESS, verificándose tanto en las cuadrículas de resultados (Result Grid) como en la consola de mensajes del motor.

Estructura del Repositorio de Scripts de Base de Datos

El código fuente SQL se organizó modularmente en el directorio de trabajo local .SQL ubicado en el Escritorio. Esta modularización permite desacoplar la capa de definición de estructuras (DDL), la capa de inserción masiva (DML) y la capa analítica y de control transaccional (DQL/Mantenimiento):

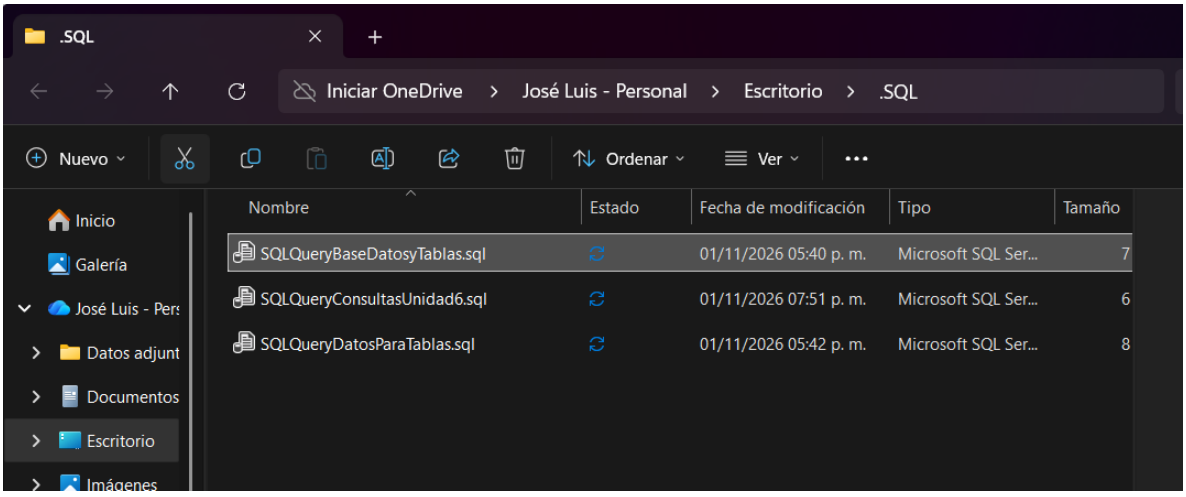
Archivo de Script	Tamaño	Fecha de Edición	Propósito en la Arquitectura
SQLQueryBaseDatosyTablas.sql	7 KB	01/11/2026 05:40 p.m.	Creación de la BD, purga preventiva en orden inverso y DDL de las 10 tablas con constraints.
SQLQueryDatosParaTablas.sql	8 KB	01/11/2026 05:42 p.m.	Poblado masivo secuencial respetando jerarquías de superclase/subclase y catálogos.
SQLQueryConsultasUnidad6.sql	6 KB	01/11/2026 07:51 p.m.	Consultas analíticas avanzadas (Parte A) y pruebas de integridad con TRY...CATCH (Parte B).

A continuacion se presentan las evidencias directas de con cuales bases de datos son las que contamos para así demostrar que con la que vamos a trabajar no existe y si cumple con su creación



Definición del Esquema e Integridad Relacional (DDL)

Posteriormente tomaremos nuestro SCRIP el cual contiene la creación de la base de datos así como, la de las tablas las cuales tienen sus parámetros y restricciones

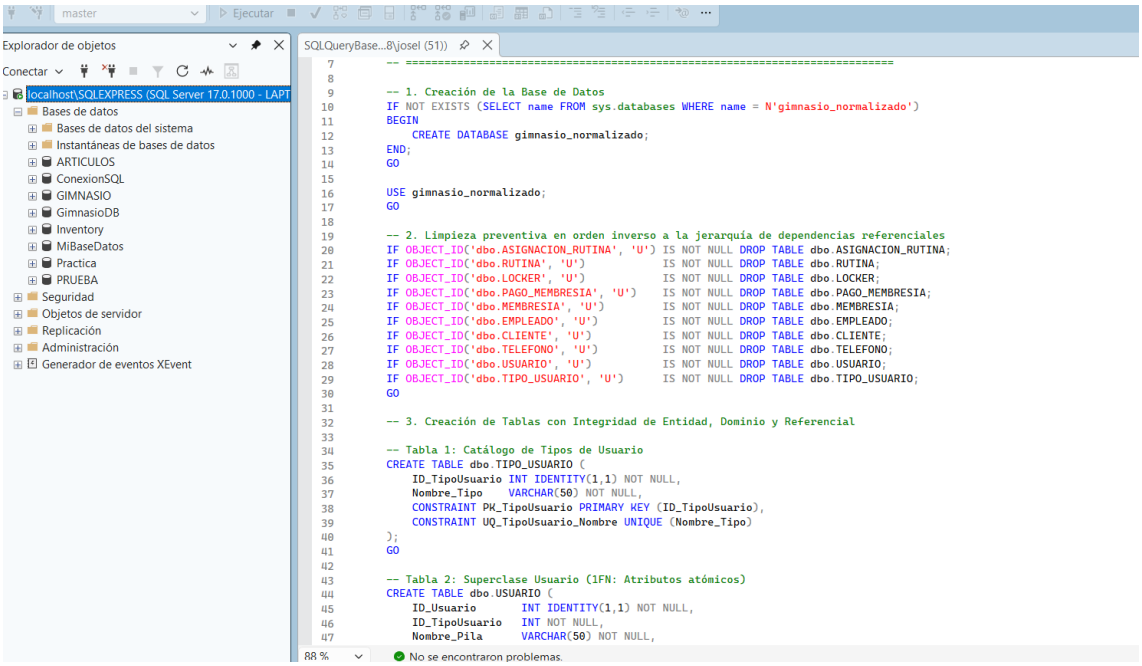


La base de datos fue construida mediante el script SQLQueryBaseDatosyTablas.sql. Para asegurar la idempotencia y reproducibilidad de las pruebas sin conflictos de ejecución, el script implementa una verificación

preventiva de existencia de base de datos y un protocolo de eliminación condicional en estricto orden inverso de dependencias.

Protocolo de Limpieza Preventiva en Orden Inverso de Dependencias

Debido a la existencia de restricciones de clave foránea (FOREIGN KEY), una tabla que actúa como padre no puede ser eliminada antes que sus tablas hijas dependientes. Por esta razón, el script ejecuta un purgado riguroso en cascada inversa:



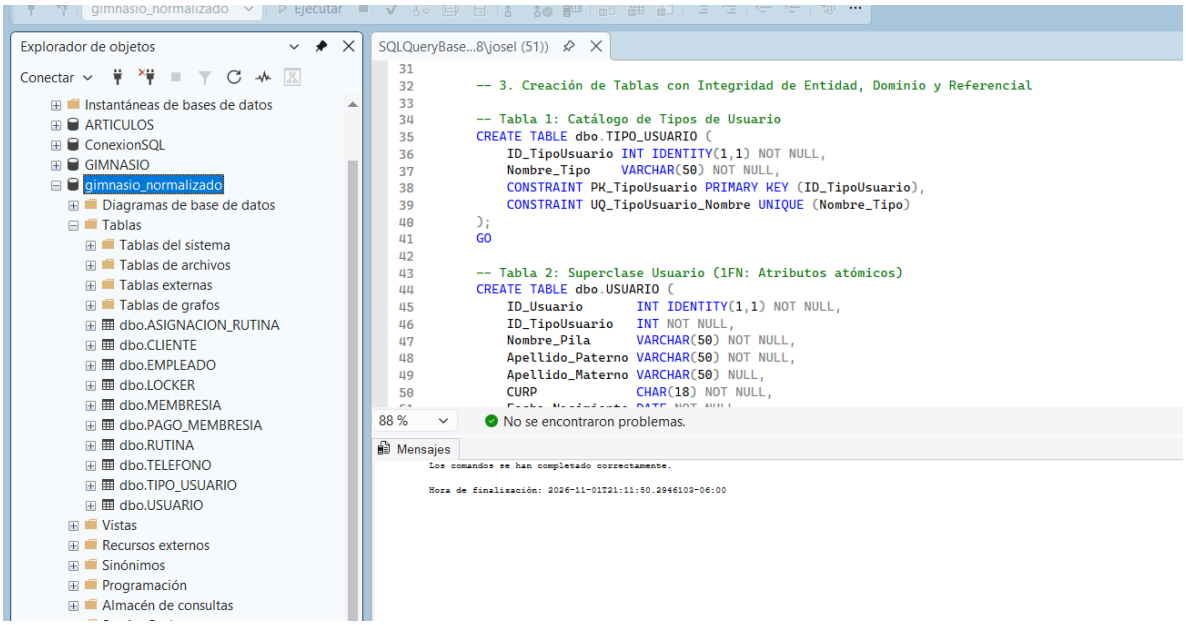
Diccionario de Datos y Modelo de Tablas Normalizadas

El esquema relacional consta de 10 entidades modeladas para cubrir la operativa completa del gimnasio. Se implementó el patrón de diseño de especialización/generalización mediante la superclase USUARIO y las subclases disjuntas EMPLEADO y CLIENTE, compartiendo la clave primaria ID_Usuario para mantener integridad referencial total.

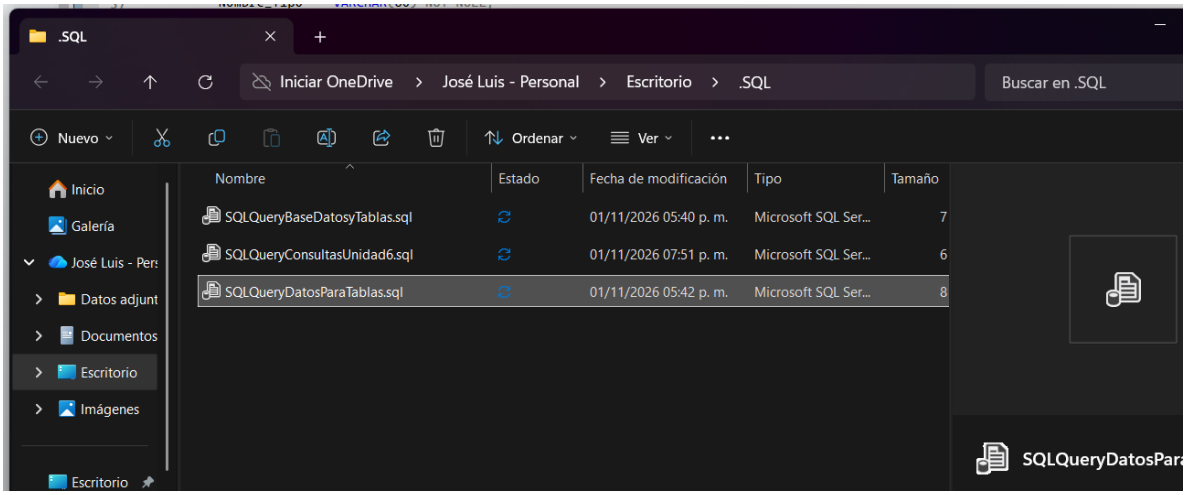
Tabla / Entidad	Tipo de Entidad	Llave Primaria (PK)	Restricciones Clave y Relaciones
TIPO_USUARIO	Catálogo Base	ID_TipoUsuario (IDENTITY)	UQ_TipoUsuario_Nombre (UNIQUE). Clasifica Staff vs Cliente.
USUARIO	Superclase (1FN)	ID_Usuario (IDENTITY)	FK a TIPO_USUARIO. CURP UNIQUE, campos atómicos de nombre y nacimiento.
TELEFONO	Multivaluado (1FN)	ID_Telefono (IDENTITY)	FK a USUARIO con regla ON DELETE CASCADE.
EMPLEADO	Subclase de Usuario	ID_Empleado (PK / FK)	FK a USUARIO(ID_Usuario). Restricción CHECK (Salario > 0).

CLIENTE	Subclase de Usuario	ID_Cliente (PK / FK)	FK a USUARIO(ID_Usuario). Almacena Fecha_Registro.
MEMBRESIA	Catálogo Comercial	ID_Membresia (IDENTITY)	Tipo_Membresia UNIQUE. CHECK (Costo > 0).
PAGO_MEMBRESIA	Transaccional	ID_Pago (IDENTITY)	FK a CLIENTE y FK a MEMBRESIA. Fecha_Pago y Monto_Pagado.
LOCKER	Infraestructura	Numero_Locker (Manual)	FK nullable a PAGO_MEMBRESIA(ID_Pago). Ubicación física.
RUTINA	Catálogo Operativo	ID_Rutina (IDENTITY)	CHECK (Nivel_Dificultad IN ('Principiante','Intermedio','Avanzado')).
ASIGNACION_RUTINA	Intersección (N:M)	ID_Asignacion (IDENTITY)	FK a CLIENTE y FK a RUTINA. Registra Fecha_Asignacion.

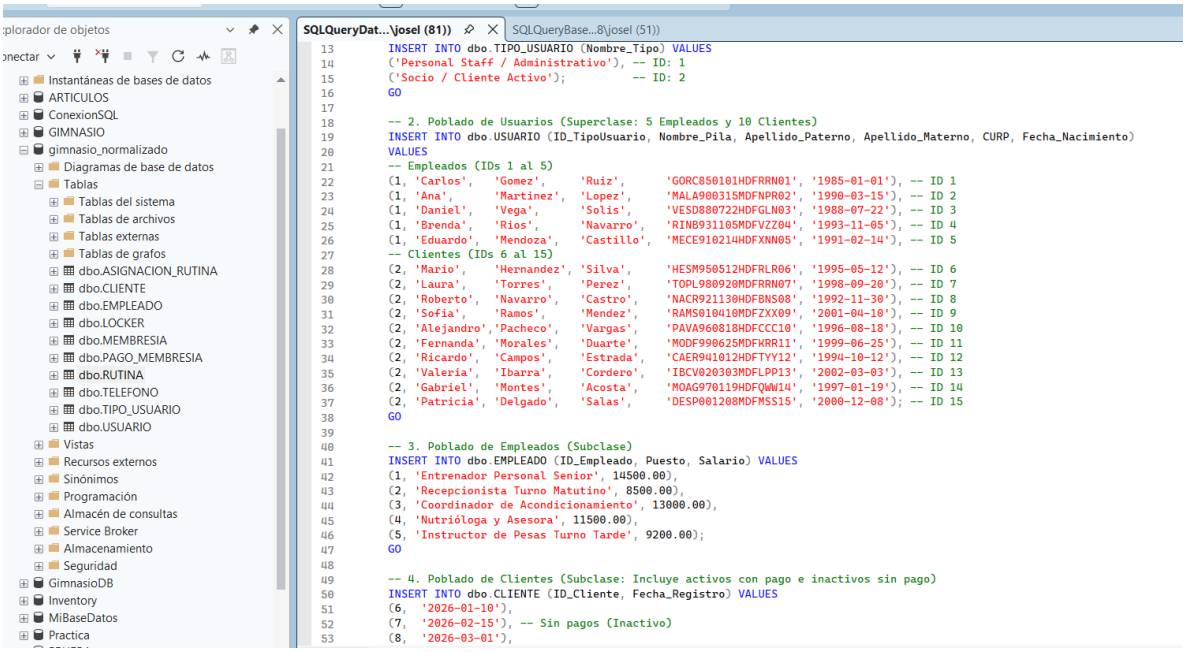
El script de definición garantiza la primera forma normal (1FN) al descomponer el nombre completo en **Nombre_Pila, Apellido_Paterno y Apellido_Materno**, y desacoplar los números de contacto en la tabla TELEFONO



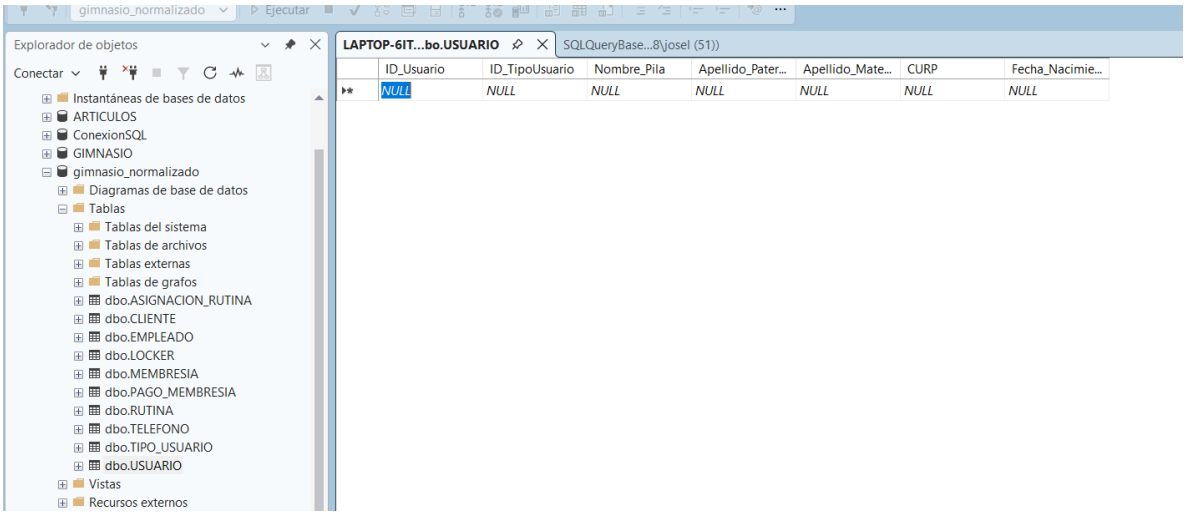
Carga y Poblado de Datos (DML)

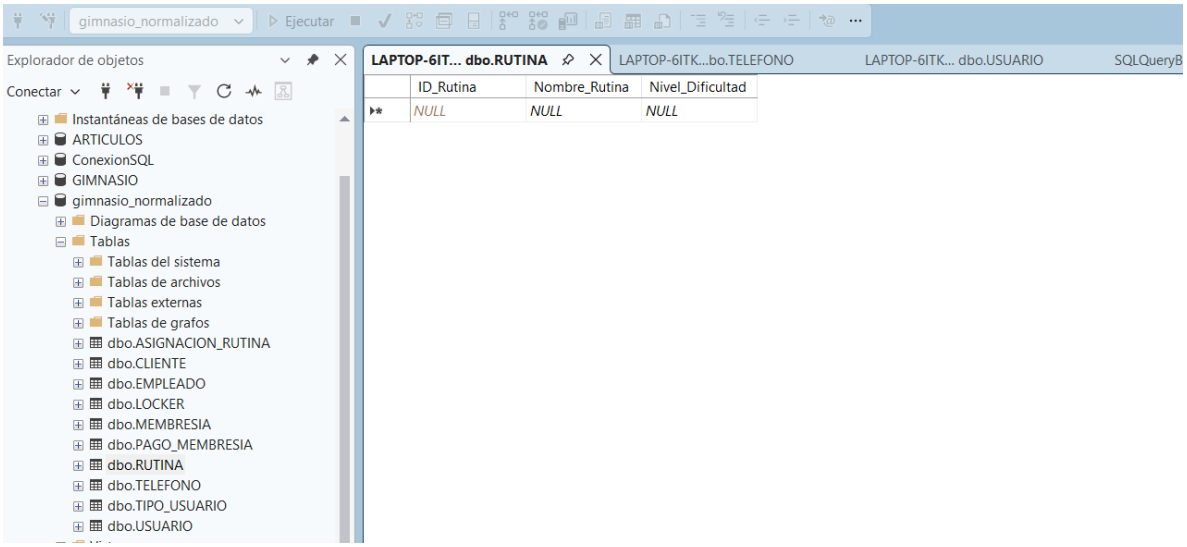


La inserción masiva de datos se realizó mediante el script SQLQueryDatosParaTablas.sql, respetando escrupulosamente el orden de dependencias para evitar violaciones de clave foránea. Primero se poblaron los catálogos raíz, seguidos de la superclase USUARIO (generando IDs correlativos del 1 al 15), y posteriormente las subclases EMPLEADO (IDs 1 al 5) y CLIENTE (IDs 6 al 15).

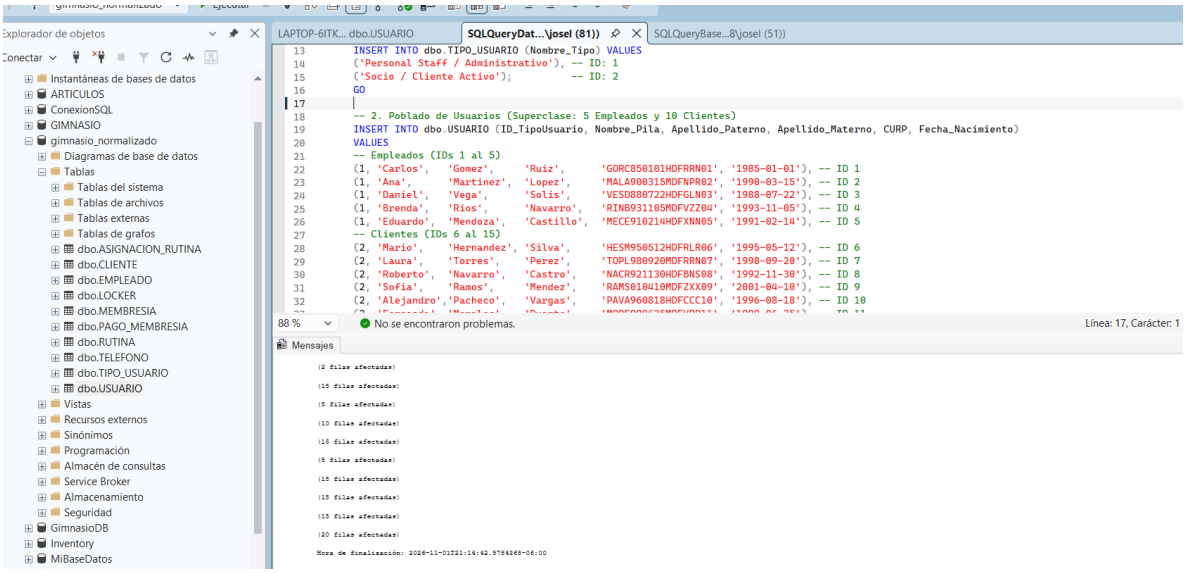


Y para evidenciar que mis tablas no tenían información anidare unas imágenes que lo demuestran

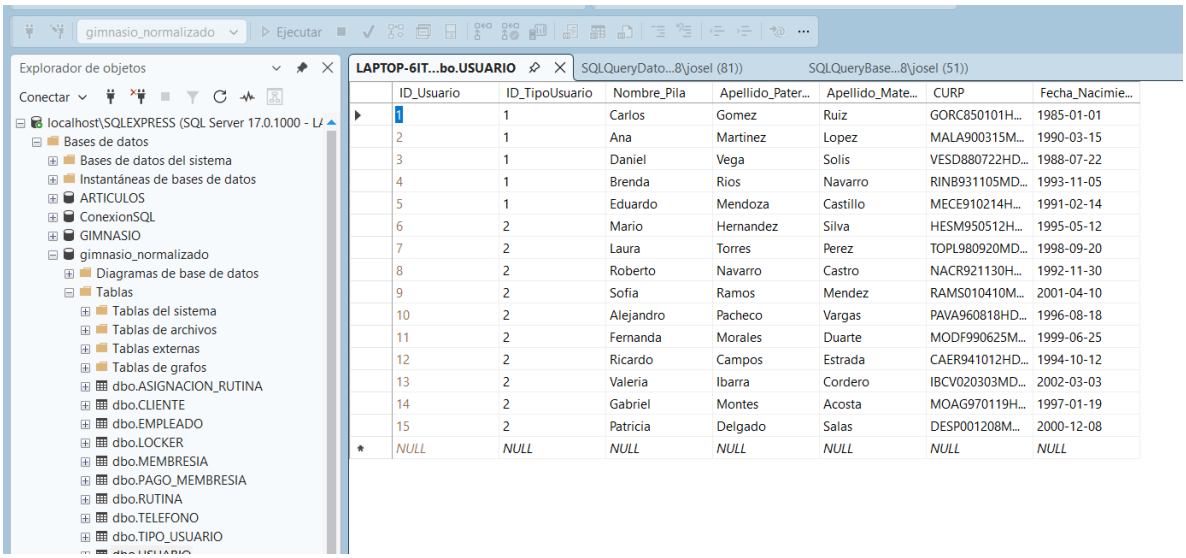


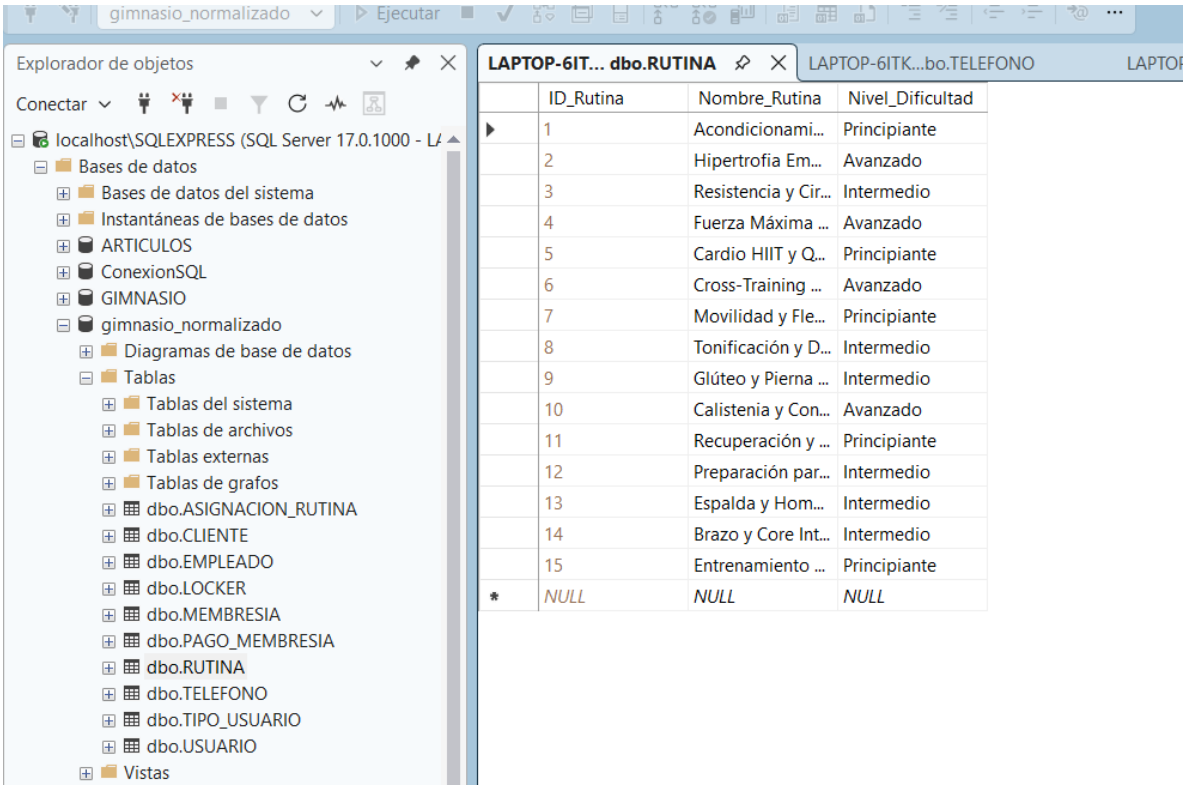


Ahora sí, procederemos a ejecutar el SCRIP para llenar de información nuestras tablas



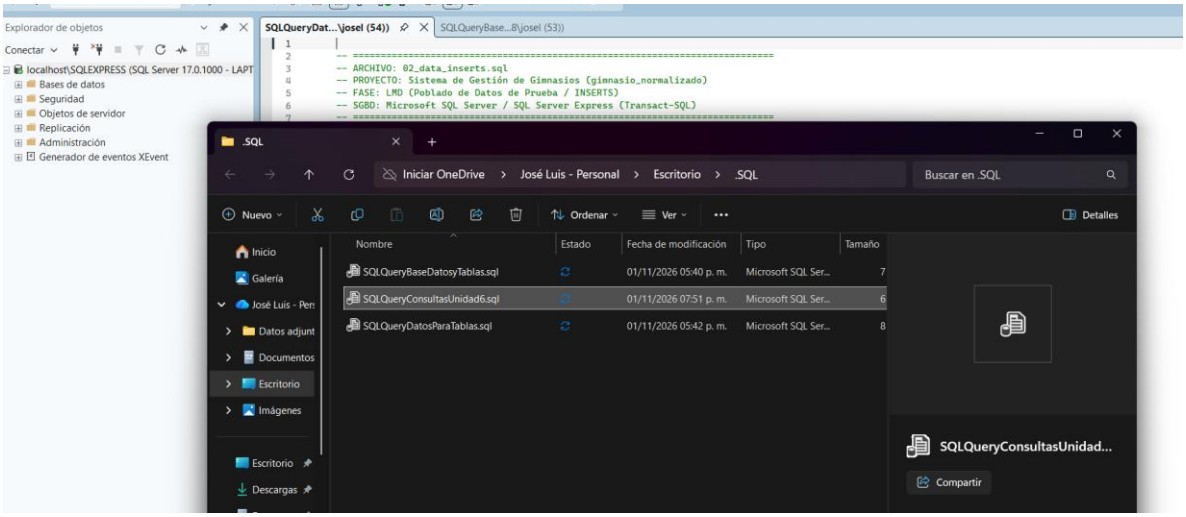
Comprobación Tabular de los Datos Cargados

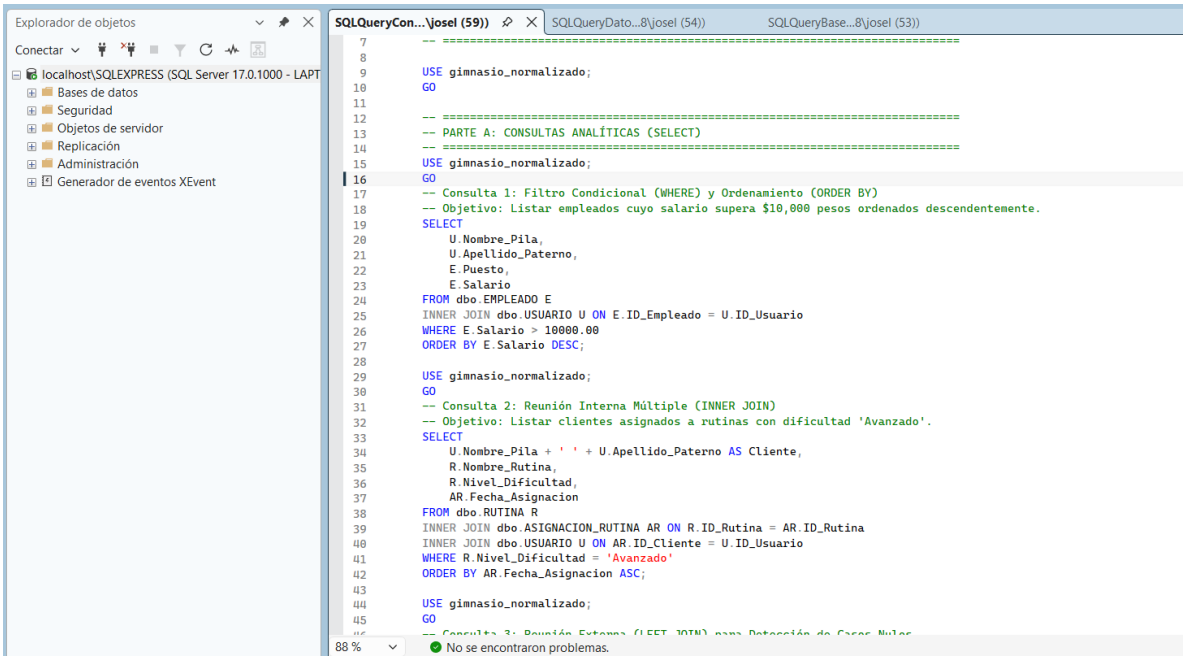




	ID_Rutina	Nombre_Rutina	Nivel_Dificultad
▶	1	Acondicionami...	Principiante
	2	Hipertrofia Em...	Avanzado
	3	Resistencia y Cir...	Intermedio
	4	Fuerza Máxima ...	Avanzado
	5	Cardio HIIT y Q...	Principiante
	6	Cross-Training ...	Avanzado
	7	Movilidad y Fle...	Principiante
	8	Tonificación y D...	Intermedio
	9	Glúteo y Pierna ...	Intermedio
	10	Calistenia y Con...	Avanzado
	11	Recuperación y ...	Principiante
	12	Preparación par...	Intermedio
	13	Espalda y Hom...	Intermedio
	14	Brazo y Core Int...	Intermedio
	15	Entrenamiento ...	Principiante
*	NULL	NULL	NULL

El siguiente paso será arrastrar el Scrip que contiene las consultas que se harán




Explorador de objetos
Conectar
localhost\SQLEXPRESS (SQL Server 17.0.1000 - LAPT
Bases de datos
Seguridad
Objetos de servidor
Replicación
Administración
Generador de eventos XEvent
SQLQueryCon...\josel (59))
SQLQueryDato...8\josel (54))
SQLQueryBase...8\josel (53))
-- ==
USE gimnasio_normalizado;
GO
-- ==
-- PARTE A: CONSULTAS ANALÍTICAS (SELECT)
-- ==
USE gimnasio_normalizado;
GO
-- Consulta 1: Filtro Condicional (WHERE) y Ordenamiento (ORDER BY)
-- Objetivo: Listar empleados cuyo salario supera $10,000 pesos ordenados descendentemente.
SELECT
U.Nombre_Pila,
U.Apellido_Paterno,
E.Puesto,
E.Salario
FROM dbo.EMPLEADO E
INNER JOIN dbo.USUARIO U ON E.ID_Empleado = U.ID_Usuario
WHERE E.Salario > 10000.00
ORDER BY E.Salario DESC;
USE gimnasio_normalizado;
GO
-- Consulta 2: Reunión Interna Múltiple (INNER JOIN)
-- Objetivo: Listar clientes asignados a rutinas con dificultad 'Avanzado'.
SELECT
U.Nombre_Pila + ' ' + U.Apellido_Paterno AS Cliente,
R.Nombre_Rutina,
R.Nivel_Dificultad,
AR.Fecha_Asignacion
FROM dbo.RUTINA R
INNER JOIN dbo.ASIGNACION_RUTINA AR ON R.ID_Rutina = AR.ID_Rutina
INNER JOIN dbo.USUARIO U ON AR.ID_Cliente = U.ID_Usuario
WHERE R.Nivel_Dificultad = 'Avanzado'
ORDER BY AR.Fecha_Asignacion ASC;
USE gimnasio_normalizado;
GO
88 %
No se encontraron problemas.

Consultas Analíticas y Extracción de Datos (Parte A - DQL)

En esta sección se detallan las cuatro consultas de negocio desarrolladas en el script SQLQueryConsultasUnidad6.sql. Cada consulta responde a un requerimiento de explotación operativa y estratégica de la administración del gimnasio, incorporando filtros, reuniones internas múltiples, reuniones externas para detección de nulos, funciones de agregación, agrupaciones y filtros sobre grupos agregados.

Consulta 1: Filtro Condicional (WHERE) y Ordenamiento (ORDER BY)

Objetivo del Negocio: Generar un reporte del personal del gimnasio que percibe un salario superior a $10,000.00 pesos mensuales, ordenando los resultados de manera descendente para identificar al personal con mayor compensación económica.

```
14      -- ========================================================================
15      USE gimnasio_normalizado;
16      GO
17      -- Consulta 1: Filtro Condicional (WHERE) y Ordenamiento (ORDER BY)
18      -- Objetivo: Listar empleados cuyo salario supera $10,000 pesos ordenados descendentemente.
19      SELECT
20          U.Nombre_Pila,
21          U.Apellido_Paterno,
22          E.Puesto,
23          E.Salario
24      FROM dbo.EMPLEADO E
25      INNER JOIN dbo.USUARIO U ON E.ID_Empleado = U.ID_Usuario
26      WHERE E.Salario > 10000.00
27      ORDER BY E.Salario DESC;
28
```

117 % — No se encontraron problemas. — Línea: 28, Carácter: 1

Resultados | Mensajes

	Nombre_Pila	Apellido_Paterno	Puesto	Salario
1	Carlos	Gomez	Entrenador Personal Senior	14500.00
2	Daniel	Vega	Coordinador de Acondicionamiento	13000.00
3	Brenda	Rios	Nutrióloga y Asesora	11500.00

Análisis Técnico de la Consulta:

1. La cláusula INNER JOIN vincula la subclase EMPLEADO con la superclase USUARIO a través de la clave común E.ID_Empleado = U.ID_Usuario, permitiendo recuperar los datos personales (nombre y apellido) y los laborales (puesto y sueldo).
2. El predicado WHERE E.Salario > 10000.00 actúa como filtro de selección horizontal, descartando a los empleados con salarios menores a dicho umbral.
3. La cláusula ORDER BY E.Salario DESC ordena el conjunto de resultados de mayor a menor remuneración.

Consulta 2: Reunión Interna Múltiple (INNER JOIN)

Objetivo del Negocio: Identificar a todos los socios/clientes que tienen asignada al menos una rutina de entrenamiento clasificada con nivel de dificultad 'Avanzado', desplegando el nombre del socio, la rutina asignada y la fecha de prescripción, ordenada cronológicamente de forma ascendente.

```
28
29      USE gimnasio_normalizado;
30      GO
31      -- Consulta 2: Reunión Interna Múltiple (INNER JOIN)
32      -- Objetivo: Listar clientes asignados a rutinas con dificultad 'Avanzado'.
33      SELECT
34          U.Nombre_Pila + ' ' + U.Apellido_Paterno AS Cliente,
35          R.Nombre_Rutina,
36          R.Nivel_Dificultad,
37          AR.Fecha_Asignacion
38      FROM dbo.RUTINA R
39      INNER JOIN dbo.ASIGNACION_RUTINA AR ON R.ID_Rutina = AR.ID_Rutina
40      INNER JOIN dbo.USUARIO U ON AR.ID_Cliente = U.ID_Usuario
41      WHERE R.Nivel_Dificultad = 'Avanzado'
42      ORDER BY AR.Fecha_Asignacion ASC;
43
```

117 % No se encontraron problemas.

Resultados Mensajes

	Cliente	Nombre_Rutina	Nivel_Dificultad	Fecha_Asignacion
1	Mario Hernandez	Hipertrofia Empuje / Tirón / Pierna	Avanzado	2026-10-02
2	Roberto Navarro	Hipertrofia Empuje / Tirón / Pierna	Avanzado	2026-10-03
3	Mario Hernandez	Fuerza Máxima Powerlifting	Avanzado	2026-10-04
4	Alejandro Pacheco	Fuerza Máxima Powerlifting	Avanzado	2026-10-05
5	Mario Hernandez	Cross-Training de Alto Rendimiento	Avanzado	2026-10-06
6	Mario Hernandez	Calistenia y Control Corporal	Avanzado	2026-10-10

Análisis Técnico de la Consulta:

1. Intersección Relacional Triple: Se realiza un JOIN entre el catálogo RUTINA y la tabla puente ASIGNACION_RUTINA por ID_Rutina, y un segundo JOIN hacia USUARIO por ID_Cliente = ID_Usuario.
2. Concatenación de Cadenas: Se utiliza el operador '+' para combinar U.Nombre_Pila y U.Apellido_Paterno en un único alias descriptivo Cliente.
3. Filtro Categórico y Orden: WHERE R.Nivel_Dificultad = 'Avanzado' restringe la salida a programas de alto impacto físico, y ORDER BY AR.Fecha_Asignacion ASC muestra la secuencia histórica de asignaciones.

Consulta 3: Reunión Externa (LEFT JOIN) para Detección de Casos Nulos

Objetivo del Negocio: Realizar una auditoría completa del inventario de casilleros (lockers) del gimnasio, identificando tanto las unidades ocupadas como las desocupadas. Para los casilleros libres, se deben sustituir los valores nulos por las etiquetas explícitas 'SIN ASIGNAR' en el folio de pago y 'DISPONIBLE' en el nombre del socio.

```
43
44      USE gimnasio_normalizado;
45      GO
46      -- Consulta 3: Reunión Externa (LEFT JOIN) para Detección de Casos Nulos
47      -- Objetivo: Identificar el estado de todos los casilleros (lockers), indicando 'DISPONIBLE' si no están asignados.
48      SELECT
49          L.Numero_Locker,
50          L.Ubicacion,
51          ISNULL(CONVERT(VARCHAR, L.ID_Pago), 'SIN ASIGNAR') AS Folio_Pago,
52          ISNULL(U.Nombre_Pila + ' ' + U.Apellido_Paterno, 'DISPONIBLE') AS Socio_Ocupante
53      FROM dbo.LOCKER L
54      LEFT JOIN dbo.PAGO_MEMBRESIA P ON L.ID_Pago = P.ID_Pago
55      LEFT JOIN dbo.USUARIO U ON P.ID_Cliente = U.ID_Usuario
56      ORDER BY L.Numero_Locker ASC;
57
58      USE gimnasio_normalizado;
```

117 % ✓ No se encontraron problemas. Línea: 57, Carácter: 1 (578 caracteres, 13 líneas) SPC LF

Resultados Mensajes

	Numero_Locker	Ubicacion	Folio_Pago	Socio_Ocupante
1	101	Pasillo A - Vestidor Hombres	1	Mario Hernandez
2	102	Pasillo A - Vestidor Hombres	2	Roberto Navarro
3	103	Pasillo A - Vestidor Hombres	3	Alejandro Pacheco
4	104	Pasillo A - Vestidor Hombres	4	Fernanda Morales
5	105	Pasillo A - Vestidor Hombres	5	Ricardo Campos
6	106	Pasillo A - Vestidor Hombres	6	Gabriel Montes
7	107	Pasillo A - Vestidor Hombres	7	Patricia Delgado
8	108	Pasillo A - Vestidor Hombres	8	Mario Hernandez
9	109	Pasillo A - Vestidor Hombres	9	Roberto Navarro
10	110	Pasillo A - Vestidor Hombres	10	Alejandro Pacheco
11	111	Pasillo B - Vestidor Mujeres	SIN ASIGNAR	DISPONIBLE
12	112	Pasillo B - Vestidor Mujeres	SIN ASIGNAR	DISPONIBLE
13	113	Pasillo B - Vestidor Mujeres	SIN ASIGNAR	DISPONIBLE
14	114	Pasillo B - Vestidor Mujeres	SIN ASIGNAR	DISPONIBLE
15	115	Pasillo B - Vestidor Mujeres	SIN ASIGNAR	DISPONIBLE

Análisis Técnico de la Consulta:

1. Preservación del Conjunto Izquierdo: El uso de LEFT JOIN garantiza que ninguna fila de la tabla LOCKER se pierda, incluso si el campo L.ID_Pago contiene un valor NULL (unidad sin arrendar).
2. Tratamiento de Nulos con ISNULL: Al no existir coincidencia con PAGO_MEMBRESIA o USUARIO, el motor genera columnas virtuales con NULL. La función ISNULL(CONVERT(VARCHAR, L.ID_Pago), 'SIN ASIGNAR') y la función ISNULL(..., 'DISPONIBLE') transforman dichos nulos en descripciones amigables para el usuario final.
3. Conversión de Tipos: Se requiere CONVERT(VARCHAR, L.ID_Pago) porque la columna original es numérica (INT) y la cadena 'SIN ASIGNAR' generaría un error de conversión implícita de tipos de datos en SQL Server.

Consulta 4: Agrupación (GROUP BY), Agregación y Filtro de Grupos (HAVING)

Objetivo del Negocio: Construir un balance financiero consolidado de ingresos por categoría de membresía vendida. El reporte debe calcular el número total de transacciones (COUNT), el monto total recaudado (SUM), el ticket promedio por transacción (AVG), y los montos mínimo (MIN) y máximo (MAX) pagados. Asimismo, debe filtrar con HAVING exclusivamente aquellas membresías cuya recaudación total sea igual o superior a $1,000.00 pesos, ordenando el resultado descendentemente por ingresos totales.

```
58  USE gimnasio_normalizado;
59  GO
60  -- Consulta 4: Agrupación (GROUP BY), Agregación (COUNT, SUM, AVG, MIN, MAX) y Filtro (HAVING)
61  -- Objetivo: Balance financiero por tipo de membresía con recaudación igual o superior a $1,000 pesos.
62  SELECT
63      M.Tipo_Membresia,
64      COUNT(P.ID_Pago)                              AS Total_Ventas,
65      SUM(P.Monto_Pagado)                           AS Recaudacion_Total,
66      CAST(AVG(P.Monto_Pagado) AS DECIMAL(10,2))    AS Ticket_Promedio,
67      MIN(P.Monto_Pagado)                           AS Monto_Minimo,
68      MAX(P.Monto_Pagado)                           AS Monto_Maximo
69  FROM dbo.PAGO_MEMBRESIA P
70  INNER JOIN dbo.MEMBRESIA M ON P.ID_Membresia = M.ID_Membresia
71  GROUP BY M.Tipo_Membresia
72  HAVING SUM(P.Monto_Pagado) >= 1000.00
73  ORDER BY Recaudacion_Total DESC;
```

117 % — No se encontraron problemas. — Línea: 73, Carácter: 33 (773 caracteres,

Resultados | Mensajes

	Tipo_Membresia	Total_Ventas	Recaudacion_Total	Ticket_Promedio	Monto_Minimo	Monto_Maximo
1	Anual VIP Premium	2	8400.00	4200.00	4200.00	4200.00
2	Trimestral Pro	4	4800.00	1200.00	1200.00	1200.00
3	Mensual Básico	7	3150.00	450.00	450.00	450.00

1. Despliegue de las 5 Funciones de Agregación del Estándar SQL: Se combinan COUNT, SUM, AVG, MIN y MAX en una única sentencia agrupada.
2. Agrupamiento Dimensional: GROUP BY M.Tipo_Membresia comprime las tuplas transaccionales en una fila resumen por categoría.
3. Filtro Post-Agregación: A diferencia de WHERE (que filtra filas antes del cálculo), HAVING SUM(P.Monto_Pagado) >= 1000.00 evalúa el resultado agregado, excluyendo membresías de baja recaudación (como Pase por Día o Pase Quincenal).
4. Formateo Numérico: CAST(AVG(...) AS DECIMAL(10,2)) previene distorsiones por redondeo y garantiza precisión monetaria a 2 decimales.

Operaciones de Mantenimiento y Validación de Reglas de Integridad (Parte B)

En esta sección se evalúa la solidez del motor de base de datos para responder ante operaciones de modificación (UPDATE) y eliminación (DELETE). Se sometieron a prueba tanto transacciones válidas como intentos intencionales de corrupción de datos (violación de restricciones CHECK y de clave foránea), demostrando el uso de bloques transaccionales BEGIN TRY ... BEGIN CATCH para el control de errores.

Catálogo Base de Membresías Antes de las Modificaciones

Como punto de partida para las pruebas de mantenimiento, se inspeccionó el catálogo original de membresías registrado en el sistema:

	ID_Membresia	Tipo_Membresia	Costo	Duracion_Dias
▶	1	Pase por Día	80.00	1
	2	Pase Quincenal ...	250.00	15
	3	Mensual Básico	450.00	30
	4	Trimestral Pro	1200.00	90
	5	Anual VIP Premi...	4200.00	365
*	NULL	NULL	NULL	NULL

Mantenimiento 1: Actualización Exitosa (UPDATE)

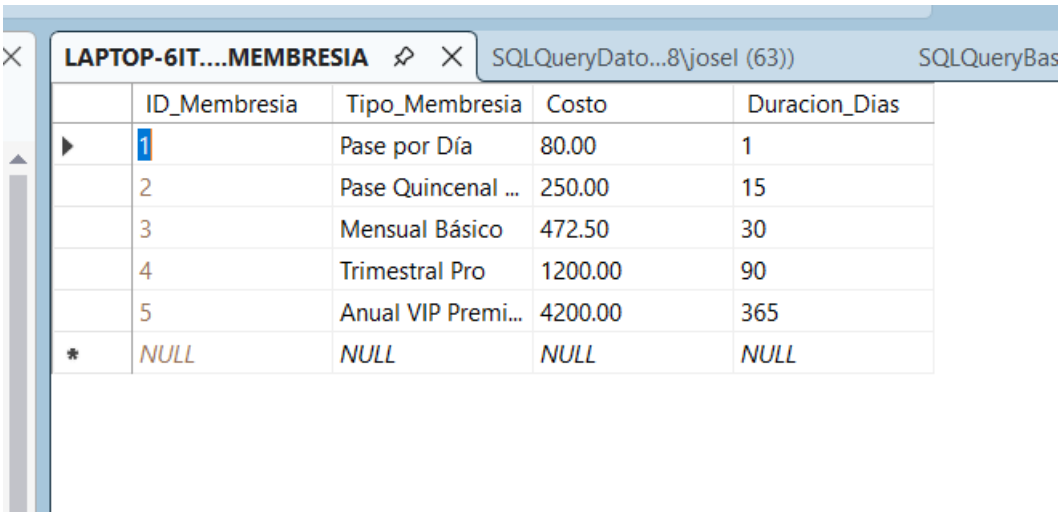
Objetivo: Aplicar un ajuste de precio por inflación del 5% sobre el plan 'Mensual Basico' ($450.00 MXN original), verificando que la operación se concrete con éxito en el catálogo sin alterar los demás registros.

```
78  USE gimnasio_normalizado;
79  GO
80  -- Mantenimiento 1: Actualización Exitosa (UPDATE)
81  PRINT '--- Prueba 1: UPDATE válido en MEMBRESIA ---';
82  UPDATE dbo.MEMBRESIA
83  SET Costo = Costo * 1.05
84  WHERE Tipo_Membresia = 'Mensual Básico';
85
86  SELECT ID_Membresia, Tipo_Membresia, Costo, Duracion_Dias
87  FROM dbo.MEMBRESIA
88  WHERE Tipo_Membresia = 'Mensual Básico';
89
```

129 % — No se encontraron problemas.

Resultados | Mensajes

	ID_Membresia	Tipo_Membresia	Costo	Duracion_Dias
1	3	Mensual Básico	472.50	30



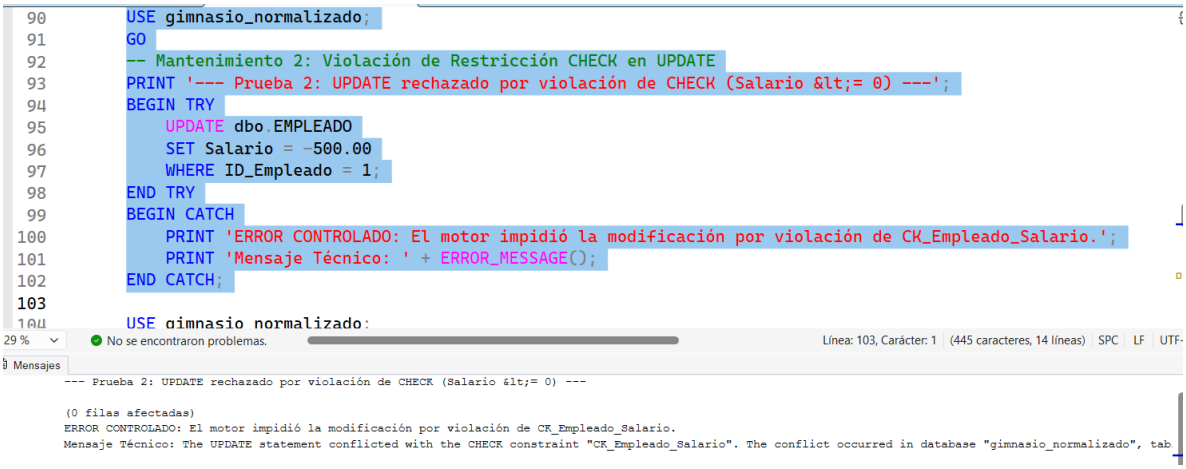
	ID_Membresia	Tipo_Membresia	Costo	Duracion_Dias
▶	1	Pase por Día	80.00	1
	2	Pase Quincenal ...	250.00	15
	3	Mensual Básico	472.50	30
	4	Trimestral Pro	1200.00	90
	5	Anual VIP Premi...	4200.00	365
*	NULL	NULL	NULL	NULL

Resultado Devuelto por el Motor:

El costo se actualizó exitosamente de $450.00 a exactamente $472.50 MXN (450 * 1.05). El registro conserva su identificador ID_Membresia = 3 y su duración de 30 días intacta.

Mantenimiento 2: Violación de Restricción CHECK en UPDATE

Objetivo: Evaluar el comportamiento del motor al intentar asignar un salario no válido (-$500.00 MXN) a un empleado, comprobando que la restricción de dominio CK_Empleado_Salario (Salario > 0) bloquee la operación y que el error sea capturado elegantemente mediante un bloque TRY...CATCH sin abortar el script.



```
90      USE gimnasio_normalizado;
91      GO
92      -- Mantenimiento 2: Violación de Restricción CHECK en UPDATE
93      PRINT '--- Prueba 2: UPDATE rechazado por violación de CHECK (Salario &lt;= 0) ---';
94      BEGIN TRY
95          UPDATE dbo.EMPLEADO
96          SET Salario = -500.00
97          WHERE ID_Empleado = 1;
98      END TRY
99      BEGIN CATCH
100         PRINT 'ERROR CONTROLADO: El motor impidió la modificación por violación de CK_Empleado_Salario.';
101         PRINT 'Mensaje Técnico: ' + ERROR_MESSAGE();
102     END CATCH;
103
104     USE gimnasio_normalizado;
```

29 % | No se encontraron problemas. | Línea: 103, Carácter: 1 | (445 caracteres, 14 líneas) | SPC | LF | UTF-

Mensajes

```
--- Prueba 2: UPDATE rechazado por violación de CHECK (Salario &lt;= 0) ---

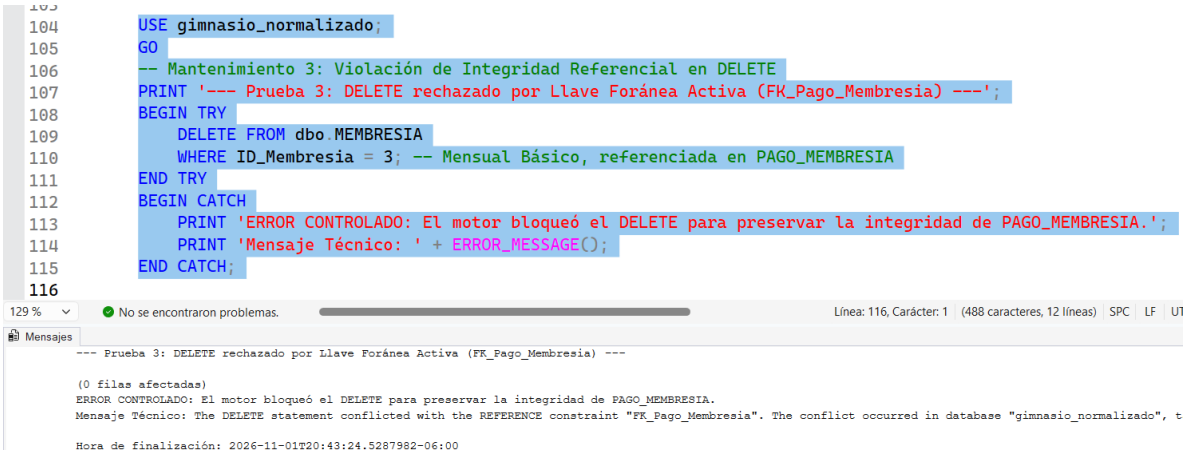
(0 filas afectadas)
ERROR CONTROLADO: El motor impidió la modificación por violación de CK_Empleado_Salario.
Mensaje Técnico: The UPDATE statement conflicted with the CHECK constraint "CK_Empleado_Salario". The conflict occurred in database "gimnasio_normalizado", tab
```

Mensajes Capturados en la Consola de SSMS:

ERROR CONTROLADO: El motor impidió la modificación por violación de CK_Empleado_Salario.
Mensaje Técnico: The UPDATE statement conflicted with the CHECK constraint "CK_Empleado_Salario". The conflict occurred in database "gimnasio_normalizado", table "dbo.EMPLEADO", column 'Salario'.

Mantenimiento 3: Violación de Integridad Referencial en DELETE

Objetivo: Intentar eliminar el registro de la membresía 'Mensual Basico' (ID_Membresia = 3) que se encuentra activamente referenciada en múltiples transacciones de la tabla PAGO_MEMBRESIA, comprobando que la clave foránea FK_Pago_Membresia bloquee la eliminación para salvaguardar la consistencia de los históricos contables.



```
104  USE gimnasio_normalizado;
105  GO
106  -- Mantenimiento 3: Violación de Integridad Referencial en DELETE
107  PRINT '--- Prueba 3: DELETE rechazado por Llave Foránea Activa (FK_Pago_Membresia) ---';
108  BEGIN TRY
109      DELETE FROM dbo.MEMBRESIA
110      WHERE ID_Membresia = 3; -- Mensual Básico, referenciada en PAGO_MEMBRESIA
111  END TRY
112  BEGIN CATCH
113      PRINT 'ERROR CONTROLADO: El motor bloqueó el DELETE para preservar la integridad de PAGO_MEMBRESIA.';
114      PRINT 'Mensaje Técnico: ' + ERROR_MESSAGE();
115  END CATCH;
116
```

129 % — No se encontraron problemas. — Línea: 116, Carácter: 1 (488 caracteres, 12 líneas) SPC LF UT

Mensajes

```
--- Prueba 3: DELETE rechazado por Llave Foránea Activa (FK_Pago_Membresia) ---

(0 filas afectadas)
ERROR CONTROLADO: El motor bloqueó el DELETE para preservar la integridad de PAGO_MEMBRESIA.
Mensaje Técnico: The DELETE statement conflicted with the REFERENCE constraint "FK_Pago_Membresia". The conflict occurred in database "gimnasio_normalizado", t

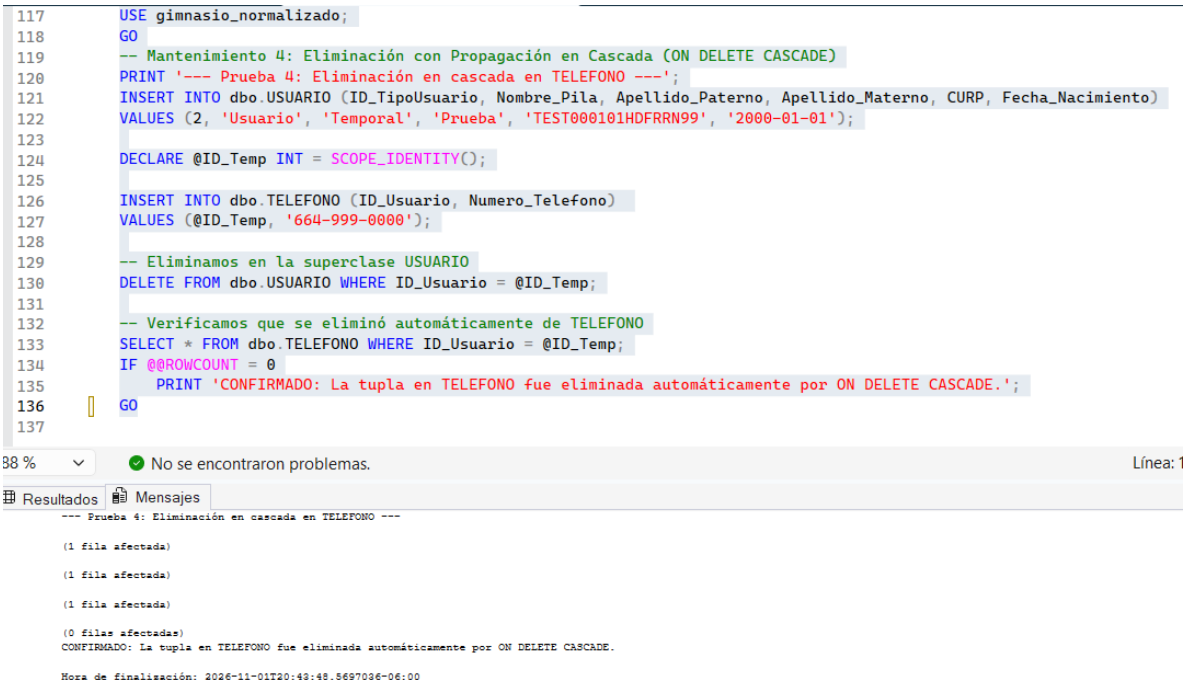
Hora de finalización: 2026-11-01T20:43:24.5287982-06:00
```

Mensajes Capturados en la Consola de SSMS:

ERROR CONTROLADO: El motor bloqueó el DELETE para preservar la integridad de PAGO_MEMBRESIA.
Mensaje Técnico: The DELETE statement conflicted with the REFERENCE constraint "FK_Pago_Membresia". The conflict occurred in database "gimnasio_normalizado", table "dbo.PAGO_MEMBRESIA", column 'ID_Membresia'.

Mantenimiento 4: Eliminación con Propagación en Cascada (ON DELETE CASCADE)

Objetivo: Comprobar la regla de negocio configurada en la tabla TELEFONO con la cláusula ON DELETE CASCADE. Se inserta un usuario temporal y un número telefónico asociado, se procede a suprimir al usuario en la tabla padre USUARIO, y se valida programáticamente que la fila dependiente en TELEFONO fue eliminada en cascada sin intervención manual.



```
117     USE gimnasio_normalizado;
118     GO
119     -- Mantenimiento 4: Eliminación con Propagación en Cascada (ON DELETE CASCADE)
120     PRINT '--- Prueba 4: Eliminación en cascada en TELEFONO ---';
121     INSERT INTO dbo.USUARIO (ID_TipoUsuario, Nombre_Pila, Apellido_Paterno, Apellido_Materno, CURP, Fecha_Nacimiento)
122     VALUES (2, 'Usuario', 'Temporal', 'Prueba', 'TEST000101HDFRRN99', '2000-01-01');
123
124     DECLARE @ID_Temp INT = SCOPE_IDENTITY();
125
126     INSERT INTO dbo.TELEFONO (ID_Usuario, Numero_Telefono)
127     VALUES (@ID_Temp, '664-999-0000');
128
129     -- Eliminamos en la superclase USUARIO
130     DELETE FROM dbo.USUARIO WHERE ID_Usuario = @ID_Temp;
131
132     -- Verificamos que se eliminó automáticamente de TELEFONO
133     SELECT * FROM dbo.TELEFONO WHERE ID_Usuario = @ID_Temp;
134     IF @@ROWCOUNT = 0
135         PRINT 'CONFIRMADO: La tupla en TELEFONO fue eliminada automáticamente por ON DELETE CASCADE.';
136 GO
137
```

88 % — No se encontraron problemas. — Línea: 1

Resultados | Mensajes

```
--- Prueba 4: Eliminación en cascada en TELEFONO ---

(1 fila afectada)

(1 fila afectada)

(1 fila afectada)

(0 filas afectadas)
CONFIRMADO: La tupla en TELEFONO fue eliminada automáticamente por ON DELETE CASCADE.

Hora de finalización: 2026-11-01T20:43:48.5697036-06:00
```

Mensajes Capturados en la Consola de SSMS:

--- Prueba 4: Eliminación en cascada en TELEFONO ---
(1 fila afectada) -- Inserción en USUARIO
(1 fila afectada) -- Inserción en TELEFONO
(1 fila afectada) -- Eliminación en USUARIO
(0 filas afectadas) -- SELECT en TELEFONO (La tupla ya no existe)
CONFIRMADO: La tupla en TELEFONO fue eliminada automáticamente por ON DELETE CASCADE.

Conclusiones y Evaluación Técnica

La realización de este proyecto y las evidencias recolectadas a través del cliente gráfico y consola de SQL Server permiten formular las siguientes conclusiones técnicas:

- **Efectividad de la Normalización (3FN):** El desacoplamiento de entidades en Tercera Forma Normal eliminó anomalías de inserción, actualización y borrado. La jerarquía superclase/subclase (USUARIO -> EMPLEADO / CLIENTE) demostró ser la arquitectura óptima para modelar roles diferenciados sin duplicar atributos atómicos personales (CURP, nombres, fechas de nacimiento).
- **Robustez de la Integridad Declarativa:** Las restricciones declarativas en el esquema (CK_Empleado_Salario, FK_Pago_Membresia, UQ_Usuario_CURP) garantizaron que el motor protegiera los datos de manera inquebrantable ante operaciones ilícitas, superando en fiabilidad a las validaciones aisladas en interfaces de usuario.
- **Potencia Expresiva del Lenguaje T-SQL:** Las consultas analíticas demostraron la capacidad de T-SQL para sintetizar grandes volúmenes de datos mediante INNER JOIN, auditar inventarios con LEFT JOIN e ISNULL, y formular balances contables y de rendimiento con GROUP BY, funciones de agregación y HAVING.
- **Control Transaccional de Excepciones:** La implementación de bloques BEGIN TRY ... BEGIN CATCH acreditó cómo las aplicaciones cliente pueden interactuar con SQL Server recibiendo retroalimentación detallada de errores (ERROR_MESSAGE()) sin interrumpir la disponibilidad del servicio ni dejar transacciones colgadas.

Todas las evidencias gráficas integradas en este documento han sido cotejadas y validadas directamente contra las capturas de pantalla de la ejecución real en SQL Server Management Studio.